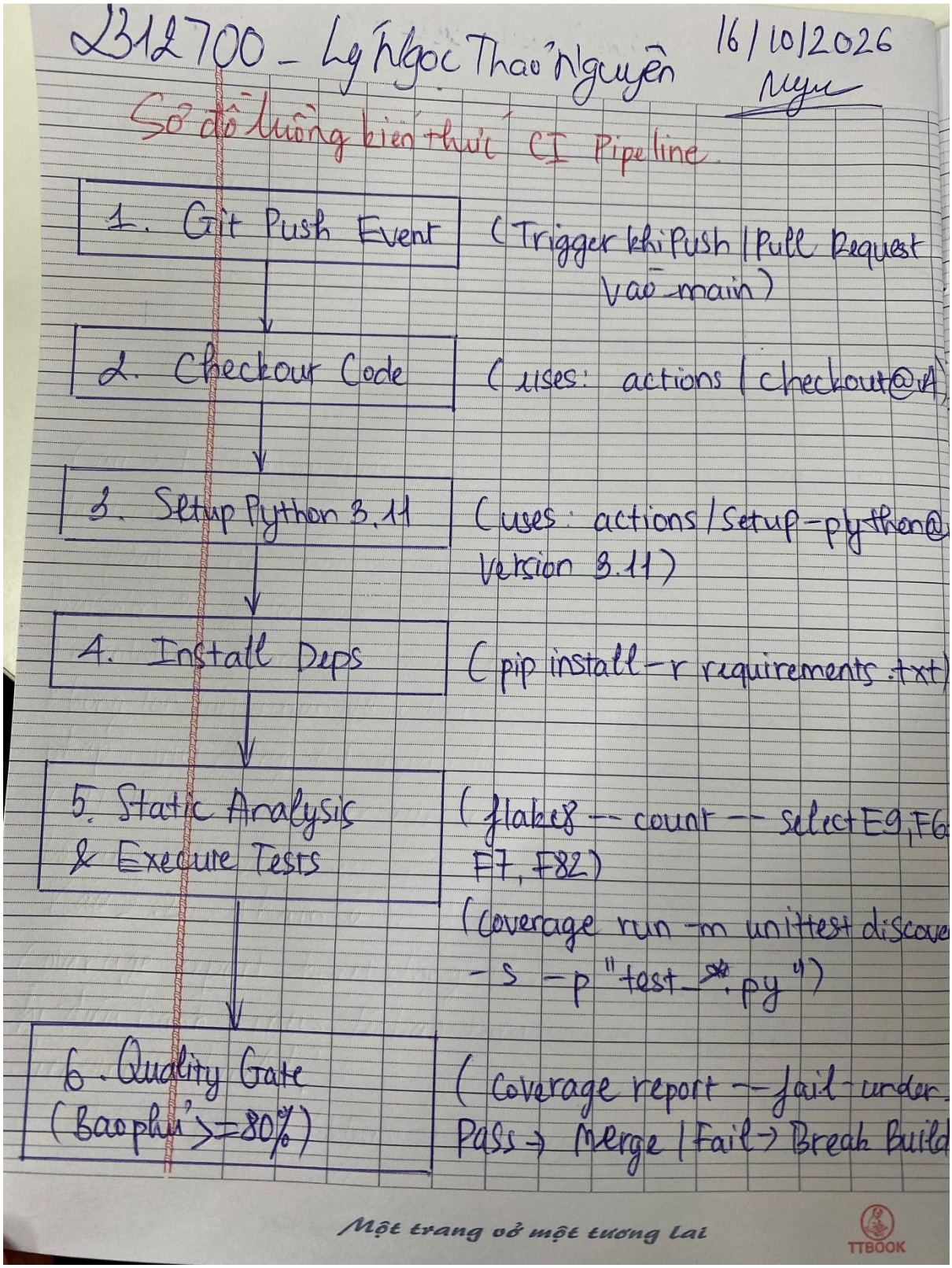
BÀI THỰC HÀNH 7. KIỂM THỬ MOCKING ỨNG DỤNG DI ĐỘNG VÀ TÍCH HỢP LIÊN TỤC (CI/CD)



23I2700 - Ly Ngoc Thao Nguyen 16/10/2026

Bảng ma trận thiết kế kịch bản Test Case (Test Matrix)

Tên Test Case	Hàm được gọi	Đối tượng và giá trị Mocking	Kết quả kỳ vọng
test_promo_online_valid	apply_promo_code	network_status = "online", os_version = 15, code = "GIAM10"	Trả về 90000.0 (Giảm 10% cho đơn 100k).
test_promo_offline_connection_error	apply_promo_code	network_status = "offline"	assertRaises (ConnectionError)
5 test_promo_new_os_valid	apply_promo_code	os_version = 15, code = "NEW_OS_ONLY"	Trả về 80000.0 (Giảm 20% cho đơn 100k)
test_promo_old_os_raises_value_error	apply_promo_code	os_version = 11, code = "NEW_OS_ONLY"	assertRaises (ValueError do HĐH < 15.
test_get_restaurants_in_range	get_nearby_restaurants	@patch ('yummy_app.gps_service.get_current_location')	Danh sách ["Nhà hàng Phở Cuốc Sĩ", "Bún Chả Hương Liên"
test_get_restaurants_8) out_of_range	get_nearby_restaurants	@patch ('yummy_app.gps_service.get_current_location') gán return (20.0, 110.0)	Trả về danh sách []

Một trang vở một dòng, cái.

TTBOOK

2312700 - Lý Ngọc Thuỷ Nguyên 16/10/2026

test_restaurants_gps_lost_error	get_nearby_restaurants	@patch('yummy_app.gps_service.get_current_location') gán side_effect= GPSSignalLostError	Trả về chuỗi "Không thể xác định vị trí."

Câu 2.1 Tại sao bước chạy Linter (flake8) phải đặt trước bước chạy test trong đường ống CI?

Đặt Linter trước tuân theo nguyên lý Fail-Fast (Thất bại sớm). Linter kiểm tra lỗi tĩnh, lỗi cú pháp (SyntaxError, IndentationError, biến chưa khai báo, ...) rất nhanh mà không tốn tài nguyên thực thi mã nguồn. Nếu mã bị lỗi cú pháp, việc dừng CI ngay lập tức giúp tiết kiệm thời gian, CPU/RAM của server runner và không mất thời gian chạy bộ unit test vô ích.

Câu 2.2 Tham số --fail-under=80 của lệnh Coverage report hoạt động theo nguyên lý nào trong hệ điều hành Linux để báo hiệu cho GitHub Actions dừng tiến trình build?

Trong Linux/Unix, mọi tiến trình sau khi kết thú

TTBOOK

2312700 - Lý Ngọc Thảo Nguyên 16/10/2026

đều trả về một mã thoát (Exit code). Mã 0 tượng trưng cho thành công (Success), còn mã khác 0 (1 đến 225) báo coverage report --fail-under=80 sẽ trả về Exit Code = 1. Runner của Github Actions kiểm tra Exit Code sau ~~khi~~ mỗi step, khi nhận mã khác 0, nó sẽ ngay lập tức ngắt tiến trình build, đánh dấu step đó màu đỏ (FAILED) và khoá đường ống.

Câu 2.3. Tham số -m trong lệnh coverage report -m hiển thị thêm thông tin gì? Giá giúp ích gì cho kỹ sư kiểm thử khi tối ưu độ bao phủ mã nguồn?

Tham số -m (viết tắt của --show-missing) hiển thị thêm cột Missing liệt kê chính xác số dòng và phạm vi các dòng lệnh (Line numbers) chưa được thực thi trong bộ test. Thông tin này giúp kỹ sư kiểm thử (SET) định vị chính xác nhánh logic hoặc hàm nào bị bỏ sót để viết bổ sung test case tập trung vào đúng các dòng mã đó nhằm đưa Coverage vượt mốc Quality Gate.

2312700 - Lý Ngọc Thiên Nguyên 16/10/2026

Câu 3.1. Phân biệt giữa khái niệm Flaky Test và lỗi do sai lệch môi trường phụ thuộc.

* Flaky Test (Kiểm thử chập chờn): Là test case trả về kết quả không ổn định (khi PASS, khi FAIL) trên cùng một môi trường mã nguồn mà không hề có sự thay đổi code nào. Nguyên nhân thường do bất đồng bộ (async / race condition), phụ thuộc vào thời gian thực, hoặc do không dọn dẹp State giữa các test case.

* Lỗi do sai lệch môi trường phụ thuộc (Environment Dependency Issue):

Là lỗi xảy ra khi test case chạy PASS ở máy local nhưng FAIL trên CI runner (hoặc người lại). Nguyên nhân do sự khác biệt về phiên bản Python, OS (Windows vs Linux), thiếu thư viện hoặc khác biệt timezone / locale.

Câu 3.2. Tại sao trên máy chủ CI bước phải "Thuần thử nguyên tắc ghim phiên bản" (Version Pinning) trong requirements.txt?

Ghim phiên bản (Sử dụng toán tử == như requests == 2.31.0) giúp đảm bảo tính đồng nhất tuyệt đối (Reproducibility) giữa môi trường phát

triển local và máy chủ CI. Nếu không ghim phiên bản, mỗi lần CI chạy nó sẽ tự động tải phiên bản mới nhất của thư viện. Khi thư viện bên thứ 3 cập nhật có breaking changes hoặc bị deprecated, pineline CI có thể sụp đổ đột ngột (Break the Build) dù mã nguồn dự án không thay đổi.

Câu 33. Trình bày quy trình 6 bước chuẩn để nâng cấp một thư viện lên phiên bản mới an toàn qua Pull Request mà không làm gián đoạn nhánh main.

1. Tạo nhánh tính năng mới (Feature Branch): Tạo nhánh mới từ main (vidu: git checkout -b update-requests-lib).

2. Cập nhật file requirements.txt: Sửa phiên bản thư viện cần nâng cấp trong file requirements.txt trên nhánh mới.

3. Kiểm thử cục bộ (Local Testing): Chạy pip install -r requirements.txt và thực thi toàn bộ bộ test suite trên máy local để đảm bảo không bị lỗi.

4. Tạo pull Request (PR): Push nhánh lên Git Hub và tạo PR gộp vào nhánh main.

5. Xác nhận tự động qua CI Pinepline: Kiểm tra GitHub Actions runner tự động kích hoạt job test test trên PR, đảm bảo tất cả Linter, UnitTest và Quality Gate đều trả về tích xanh (PASS)

6. Review và merge: Mỗi đồng nghiệp review code, phê duyệt PR và tiến hành gộp (merge) an toàn vào nhánh main.